ML PIPELINE FORMATIVE 1

Building a Pipeline for Time Series Data

Machine Learning Pipeline · ALU · 2026

Group Members

Gaddiel Irakoze · Gabriel Tuyisingize Sezibera · Etienne Kagaba · Sheilla Keza Ruvugabigwi

GitHub Repository: *https://github.com/kagaba-etienne/ml-pipeline-formative-1.git*

Contribution Tracker: **BSE Group Assignments : Task Sheet_[ML Pipeline_Formative 1_Cohort 1_Team 2]**

Project Overview

This report covers our implementation of Formative 1 for the Machine Learning Pipeline course at ALU. The work splits across four tasks: time-series exploratory analysis and machine learning modelling (Task 1), relational and non-relational database design (Task 2), a CRUD API with time-series query support (Task 3), and a forecasting script (Task 4).

The dataset is a set of daily stock records for five tickers: AAPL (Apple), CAT (Caterpillar), JPM (JPMorgan Chase), MCD (McDonald's), and WMT (Walmart), covering January 2006 to December 2017.

One disclaimer up front: this project does not make investment recommendations. Our focus is the prediction problem itself, building and evaluating machine learning models that forecast future values from historical patterns. Any results here are for academic and technical evaluation only and are not financial advice.

Item	Detail
Dataset	Daily stock prices: AAPL, CAT, JPM, MCD, WMT (2006-2017)
Records	15,095 rows across 5 stocks after cleaning
ML Model	Random Forest + Linear Regression
Databases	PostgreSQL (primary) + MongoDB (secondary)
API Framework	FastAPI + Prisma ORM
Contributors	Gaddiel, Gabriel, Etienne, Sheilla

Table 1: Project summary

Contributor Responsibilities

Contributor	Task
Gaddiel	Task 1: EDA, feature engineering, ML modelling
Gabriel	Task 2: Database design; Task 3: API endpoints
Etienne	Task 2: DB config & loading scripts; Task 4: Forecasting script
Sheilla	Task 4: Prediction coordination; final deliverables and report

Table 2: Contributor responsibilities

Literature Review

1. Time-Series Forecasting in Financial Markets

Stock price prediction has been a long-standing problem in quantitative finance and machine learning. Fama (1970), in his work on the Efficient Market Hypothesis, argued that asset prices already reflect all available information, which would mean future prices cannot be predicted from historical data alone. Later empirical work found persistent short-term autocorrelation in daily returns, and that is what motivates using lagged price features as predictors.

Box and Jenkins (1976) formalised ARIMA models for time-series forecasting, which set the theoretical basis for lag-based prediction. More recent work applies machine learning to financial forecasting, and a growing literature compares classical statistical models against ensemble methods and deep learning.

2. Machine Learning for Stock Price Prediction

Random Forest, introduced by Breiman (2001), is widely used for financial prediction because it resists overfitting and handles high-dimensional feature spaces well. Khaidem et al. (2016) showed that Random Forest classifiers trained on technical indicators such as moving averages, RSI, and lagged returns can reach useful accuracy on daily stock data. They also noted that tree-based ensembles struggle to extrapolate beyond the range of values seen in training, which is a real problem for trending financial series.

Linear regression, despite its simplicity, can compete in financial forecasting when the predictors have a strong linear relationship with the target. Atsalakis and Valavanis (2009) reviewed over 100 papers on computational intelligence methods for stock forecasting and found that simple linear models often held up as strong baselines, especially over short horizons where autocorrelation is the main signal.

3. Feature Engineering for Financial Time Series

Good features are one of the most important parts of financial machine learning. Prado (2018), in Advances in Financial Machine Learning, stresses avoiding data leakage by splitting chronologically and using purged cross-validation. He also points to lagged prices, rolling statistics, and calendar effects as core features for financial ML pipelines.

Moving averages are among the most common technical indicators in both research and practice. The 7-day moving average we use captures short-term momentum while smoothing out day-to-day noise. Murphy (1999), in Technical Analysis of the Financial Markets, covers the moving-average crossover strategies that much of the feature-engineering literature builds on.

4. Database Design for Time-Series Data

Storing financial time-series data means thinking about both schema design and query performance. Fowler (2012), in NoSQL Distilled, works through the trade-offs between relational databases (built for structured, normalised data with complex joins) and document stores (built for flexible, hierarchical records with high write throughput). For financial OHLCV data, relational schemas with a compound index on (stock_id, trade_date) are a well-established pattern for fast range queries.

MongoDB has seen more use in financial data pipelines thanks to its flexible schema and native time-series collections (added in version 5.0). Its aggregation pipeline, with operators such as $setWindowFields for rolling-window calculations, lets you run time-series analytics inside the database instead of in application code.

5. RESTful API Design for Data Pipelines

FastAPI has become a popular framework for building fast REST APIs in Python, especially for machine learning and data engineering. Its async request handling, automatic OpenAPI docs, and Pydantic request validation make it a good fit for serving time-series data with complex filters. Ramalho (2021), in Fluent Python, covers the type annotations and validation patterns that FastAPI's design leans on.

Problem Definition and Dataset Justification

1. Problem Statement

Stock prices move constantly, reacting to economic conditions, company results, investor sentiment, and global events. That makes future movements hard to predict with any confidence.

Better forecasting helps. If you can read historical trends, you can make more informed investment and risk decisions. The problem is that simple methods tend to miss the time-dependent patterns in financial data.

So we take a data-driven approach. Using historical prices and engineered features like lagged values and moving averages, we try to capture those patterns and forecast the next day's price more accurately.

2. Dataset Relevance to the Problem

The dataset fits this problem well. It holds daily records for five large public companies, Apple (AAPL), Caterpillar (CAT), JPMorgan Chase (JPM), McDonald's (MCD), and Walmart (WMT), from January 2006 to December 2017. Each trading day gives the open, high, low, and close prices, the trading volume, and the company name.

Together these columns make up a multivariate time series we can use to study trends, volatility, and how each day depends on the ones before it. The daily closing price is the forecasting target. Past prices, volume, lagged values, and moving averages are the features. Because the five companies sit in different sectors, we can also compare how stocks behave across industries and check whether the same methods carry over from one to another.

Task 1: Time-Series Preprocessing and Exploratory Analysis

A. Understanding the Dataset

The analysis used dataset/stocks_dataset.csv, built by combining the five ticker CSVs from January 2006 to December 2017. We sorted records by Date and Name. Three rows had null High/Low values and four had null Open values, so we dropped them with dropna(). That is under 0.03% of the data, so removal was the safe choice. The granularity is daily: one record per stock per trading day.

Property	Value
Date range	3 January 2006 to 29 December 2017

Total records	15,095 rows (after cleaning)
Granularity	Daily: one record per stock per trading day
Tickers	AAPL, CAT, JPM, MCD, WMT
Columns	Date, Open, High, Low, Close, Volume, Name

Table 3: Dataset properties

B. Analytical Questions

Q1: Long-term price trend across all stocks

All five stocks grew over the long run between 2006 and 2017, with a shared dip during the 2008 to 2009 financial crisis. Apple grew the fastest and swung the most; Walmart stayed the most stable across the period.

Figure 1: Closing price trajectories for all five stocks, 2006-2017

Q2: Which stock is the most volatile?

We measured volatility as the standard deviation of each stock's daily closing price. Apple (AAPL) had the highest, which makes it the most volatile of the five. Walmart (WMT) had the lowest, which fits its defensive business.

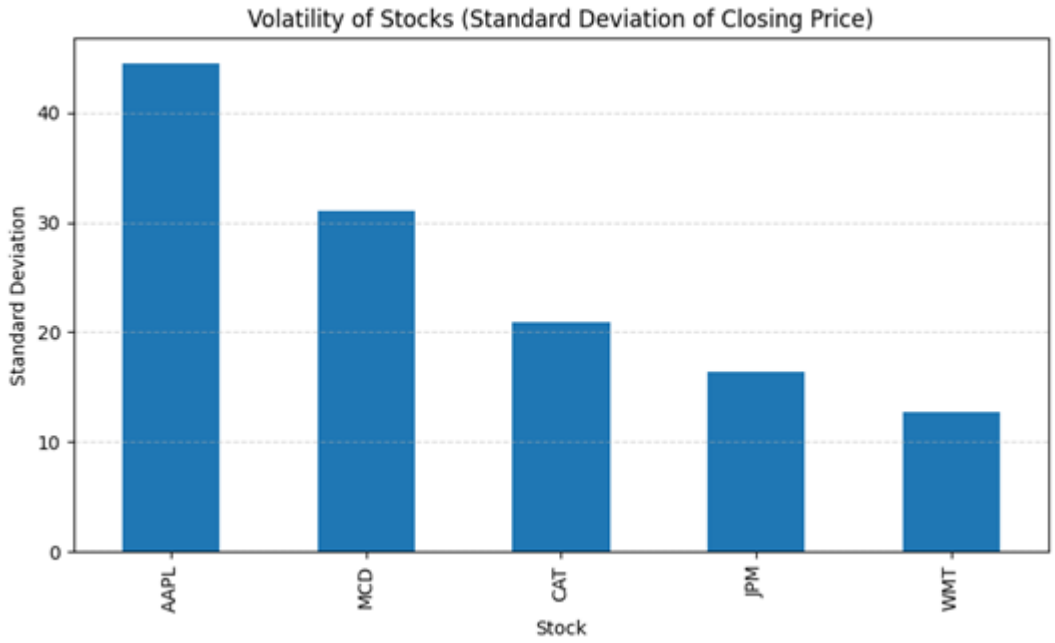

Figure 2: Volatility comparison (standard deviation of closing price)

Q3: Do the stocks correlate with each other?

Every pairwise correlation was positive, from 0.58 (CAT and JPM) to 0.93 (AAPL and MCD). That points to broad market forces moving all five at once, which limits the diversification benefit of holding them together.

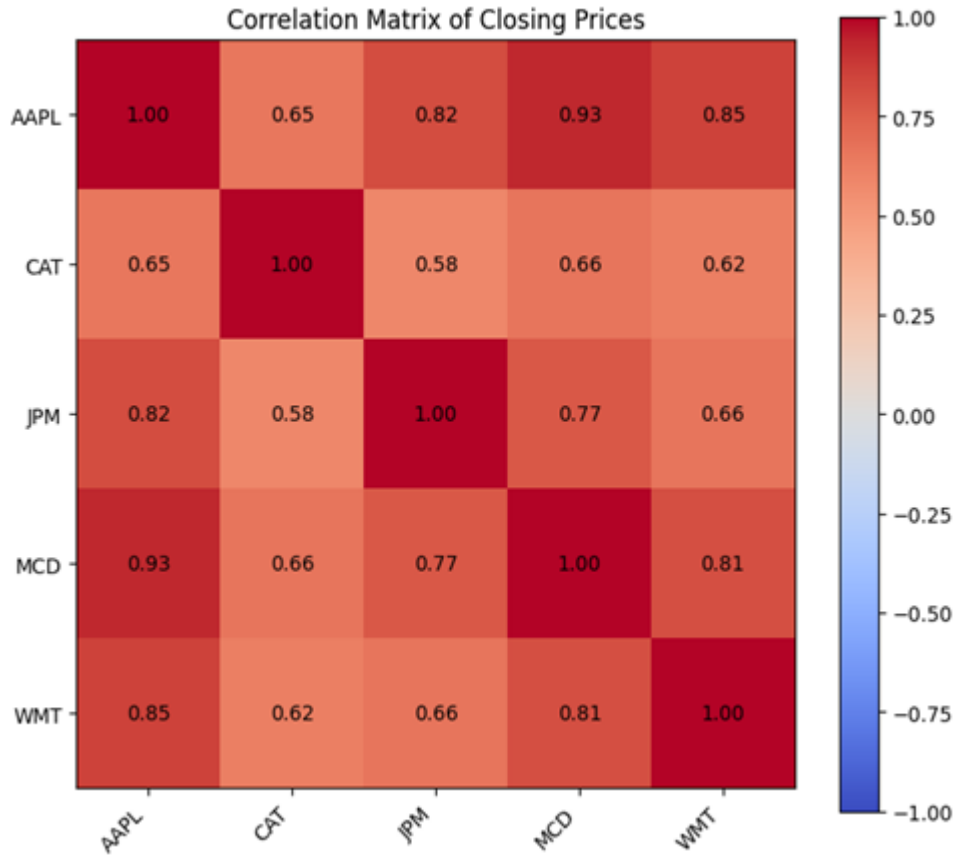

Figure 3: Correlation heatmap of closing prices across all five stocks

Q4 (Lag Feature): Does yesterday's close predict today's close?

We built a Lag1 feature by shifting each stock's Close back by one day. The scatter plots for all five show a tight positive linear relationship between Lag1 and the current day's Close. That confirms strong autocorrelation and makes Lag1 a primary predictor.

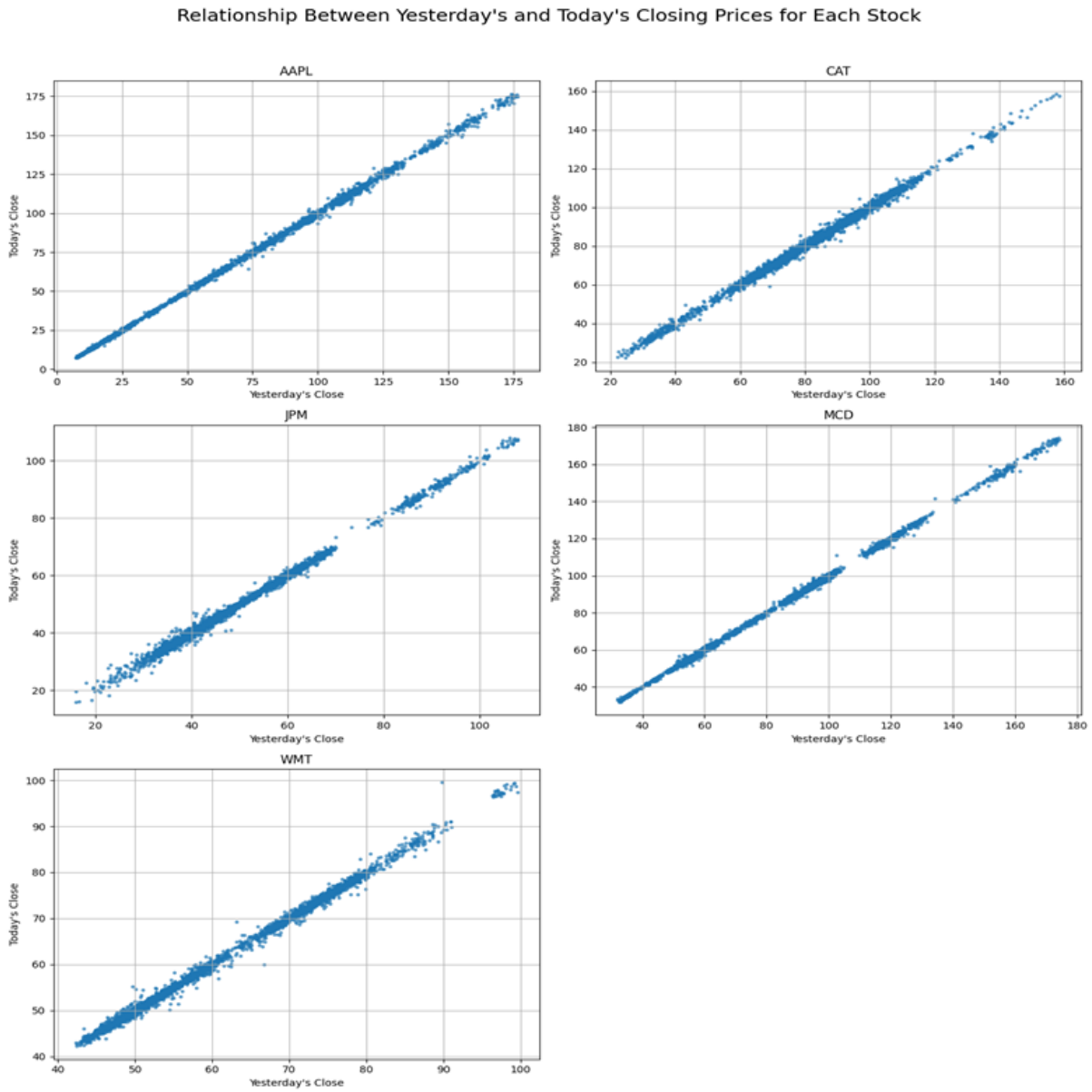

Figure 4: Lag1 (yesterday's close) vs today's close for each stock

Q5 (Moving Average): How does the 7-day MA compare to actual prices?

We computed a 7-day moving average (MA7) per ticker with a rolling window. Plotted against Apple's actual close, MA7 smooths the daily noise while still tracking the trend. When the price moves fast, MA7 lags a little, which is expected for a backward-looking smoother.

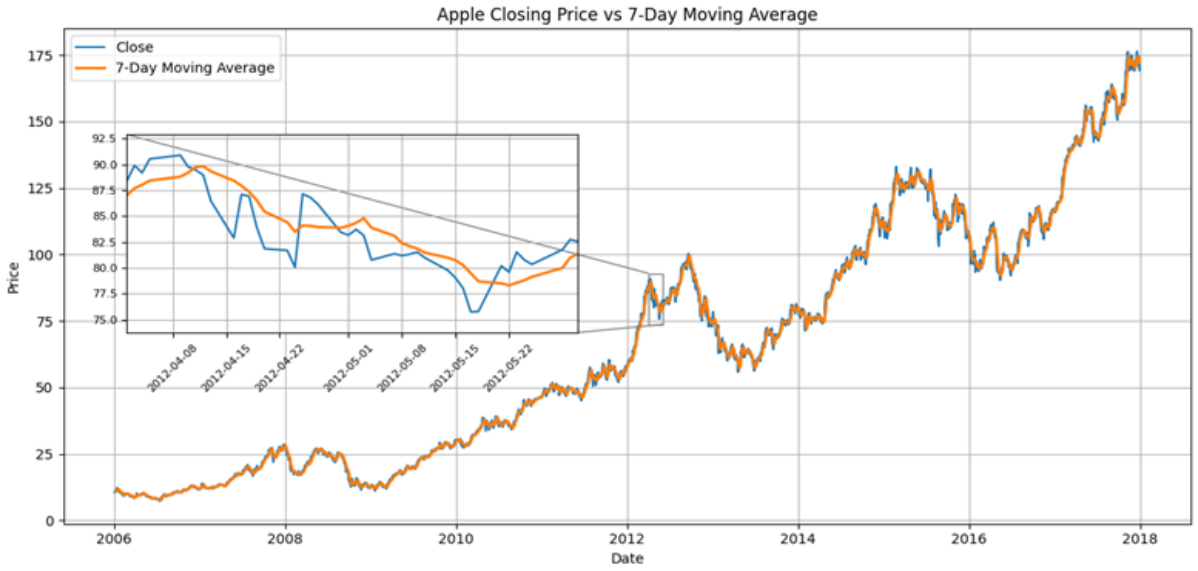

Figure 5: Apple closing price vs 7-day moving average (with zoomed inset, April to May 2012)

C. Machine Learning Models

The prediction target is the next trading day's closing price (Close.shift(-1) per ticker), which avoids data leakage. We split the data chronologically: the first 80% for training and the last 20% for testing. The features are Open, High, Low, Volume, Lag1, MA7, Year, Month, Day, DayOfWeek, and one-hot ticker dummies (reference: AAPL).

Random Forest Model: Results

The two Random Forest experiments (Experiment 1 with default parameters, Experiment 2 tuned with GridSearchCV) performed about the same. Tuning helped only a little: MAE dropped from 4.5522 to 4.4480 and R^2 rose from 0.878 to 0.883. On their own those numbers look fine, but the actual-vs-predicted plot shows the real problem. The Random Forest predictions flatten out at the highest prices seen in training, so they hit a ceiling on the test set while actual prices kept climbing. This is the known extrapolation limit of tree-based ensembles.

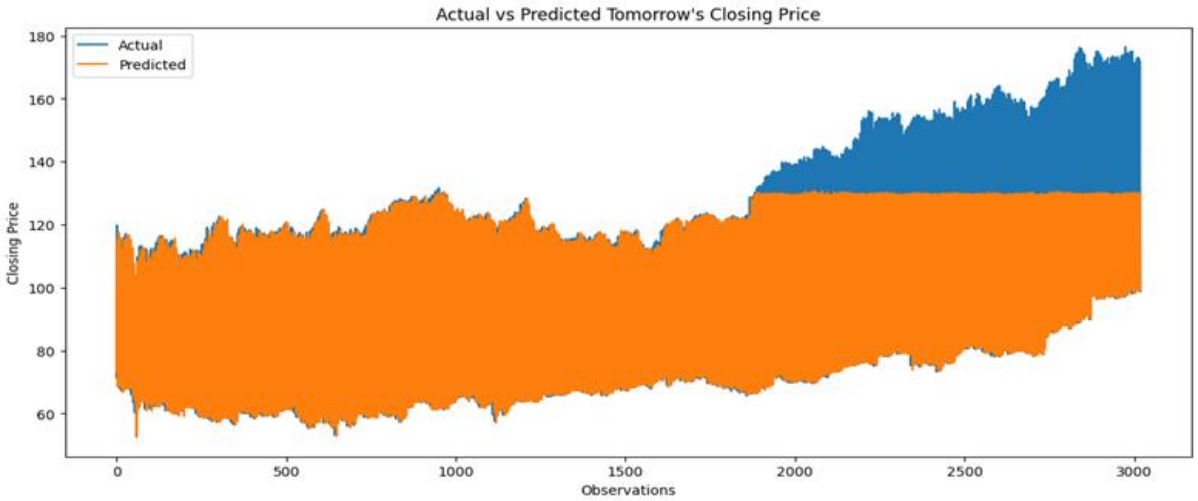

Figure 6: Random Forest: actual vs predicted next-day closing price over the test set

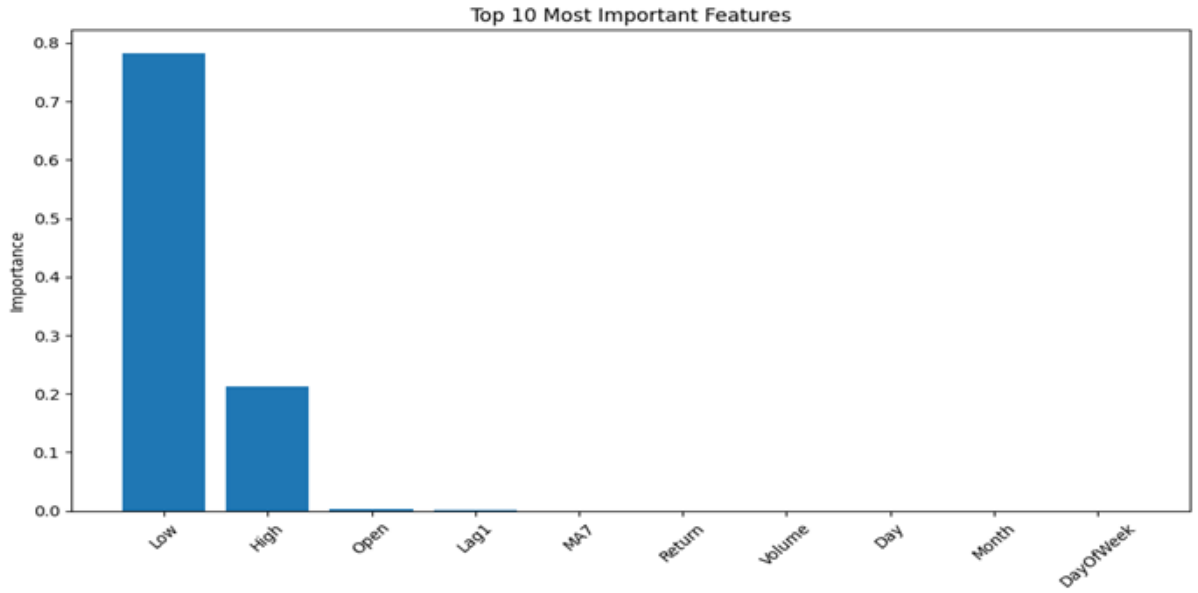

Figure 7: Random Forest feature importances: Close and Lag1 dominate

Linear Regression Model: Results

Linear Regression clearly beat both Random Forest variants. It reached an R^2 of 0.998 and cut the MAE to 0.896, about five times lower than the best Random Forest result. Its actual-vs-predicted plot lines up almost exactly across the whole test set, including the higher prices the Random Forest missed. Looking at the coefficients, Lag1 (about 0.82) and MA7 (about 0.18) explain nearly all of the target's variance, and the other features barely move the prediction. Because it has no extrapolation ceiling, Linear Regression fits trending financial series much better.

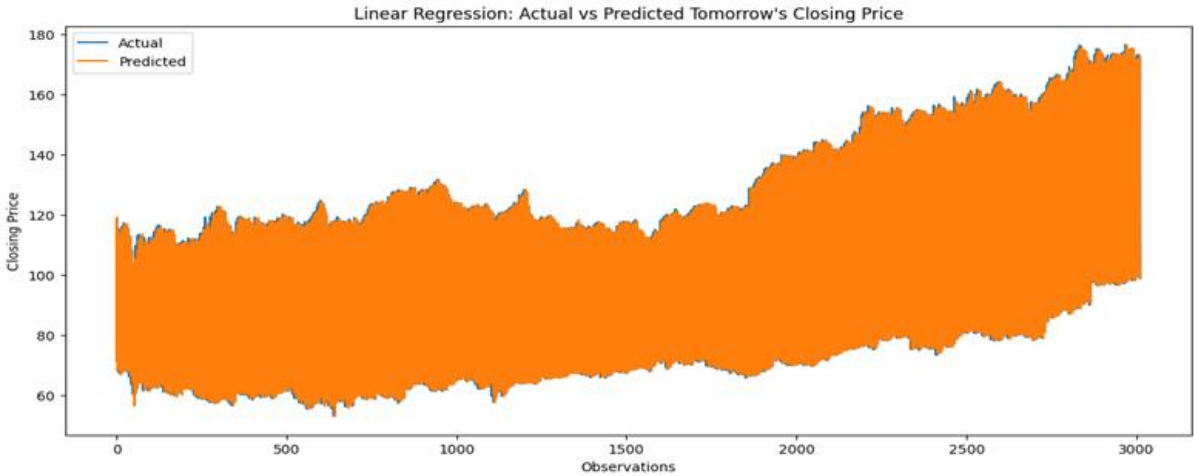

Figure 8: Linear Regression: actual vs predicted next-day closing price over the test set

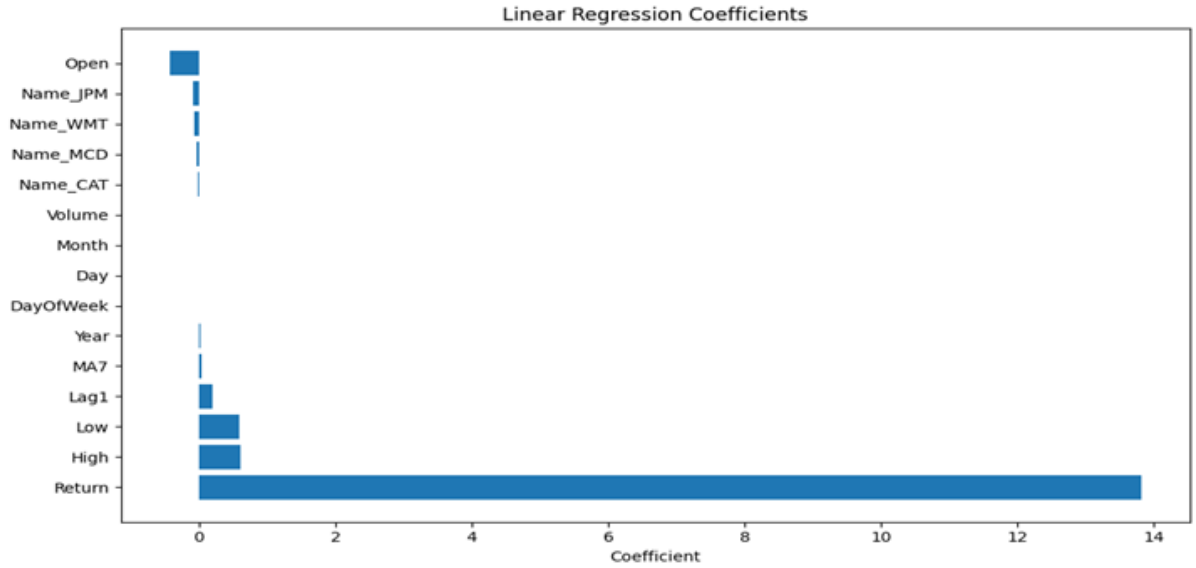

Figure 9: Linear Regression coefficients: Return, High, and Low dominate alongside Lag1

Model Comparison Summary

Side by side, the main difference between the two families is how well they handle price levels they never saw in training. The Random Forest models, for all their complexity, overfit the training range and broke down once test prices went past it. Linear Regression instead caught the main linear link between lagged prices and the next-day target, and it held up across the full price range. That matches the financial-forecasting literature, which finds that simple models often beat complex ones on trending series where autocorrelation dominates.

Experiment Results

Experiment	Model	MAE	RMSE	R^2
Experiment 1	Baseline Random Forest (default params)	4.5522	10.7120	0.878
Experiment 2	Tuned Random Forest (GridSearchCV)	4.4480	10.4961	0.883
Experiment 3	Linear Regression (default params)	0.8962	1.3148	0.998

Table 4: Model performance comparison across three experiments

Task 2: Database Design (PostgreSQL and MongoDB)

A. PostgreSQL Design

The relational schema is defined in config/database/postgres/prisma/schema.prisma and the full SQL DDL with seed data is in config/database/postgres/seed.sql. The schema consists of three normalised tables that enforce referential integrity through foreign key constraints between sectors → stocks → stock_prices.

Table	Primary Key	Purpose
sectors	sector_id (SERIAL)	Stores sector metadata including sector name and description. Groups stocks by economic sector (Technology, Industrials, Financials, Consumer Staples).
stocks	stock_id (SERIAL)	Stores stock-level metadata: ticker symbol, company name, the exchange it is listed on, and a foreign key linking it to its sector.
stock_prices	price_id (BIGSERIAL)	The core time-series table. Stores one row per stock per trading day, containing the full OHLCV record (open, high, low, close prices and trading volume).

Table 5: PostgreSQL table descriptions

Key constraints: UNIQUE(stock_id, trade_date) on stock_prices prevents duplicate daily records; an INDEX on trade_date and a compound INDEX on (stock_id, trade_date) optimise time-series range queries and per-stock lookups respectively.

PostgreSQL Entity-Relationship Diagram

The diagram below shows the three tables and their relationships, as generated from the repository's config/database/postgres/erd_diagram.png. Each sector contains multiple stocks, and each stock has many daily price records.

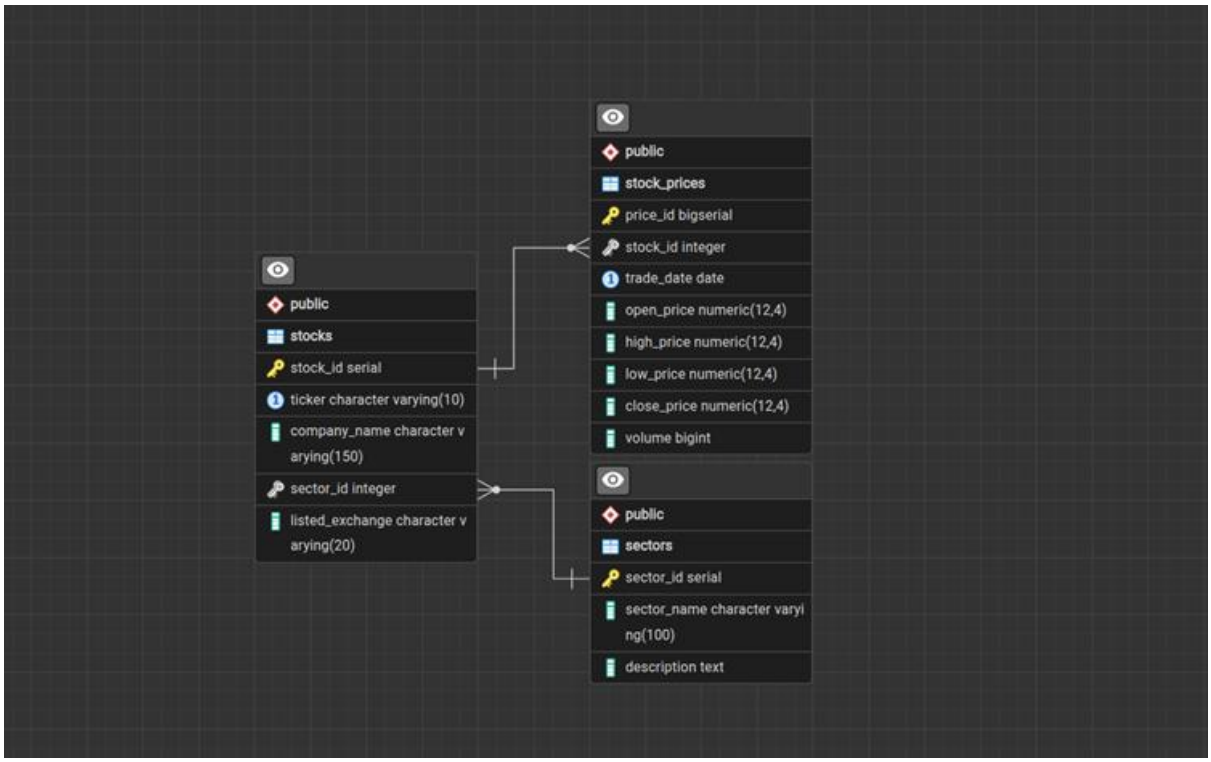

Figure 10: PostgreSQL Entity-Relationship Diagram (sectors → stocks → stock_prices)

PostgreSQL Schema Overview

Entity	Fields	Cardinality

SECTORS	sector_id PK, sector_name, description	1 sector → many stocks
STOCKS	stock_id PK, ticker, company_name, sector_id FK, listed_exchange	1 stock → many price records
STOCK_PRICES	price_id PK, stock_id FK, trade_date, open_price, high_price, low_price, close_price, volume	Each row = 1 trading day per stock

Table 6: PostgreSQL entity relationships

B. MongoDB Design

The MongoDB schema is defined in config/database/mongo/prisma/schema.prisma with seed data in config/database/mongo/seed.js. Three collections mirror the relational structure, with ObjectId references replacing foreign keys. A compound index on { stock_id: 1, trade_date: -1 } on the stock_prices collection optimises the most common query pattern: recent price history per stock ordered by date descending.

Collection	Key Fields	Reference	Purpose
sectors	_id, name, description	None	Stores sector classification metadata, equivalent to the SQL sectors table.
stocks	_id, ticker, company_name, listed_exchange, sector_id	sector_id → sectors._id	Stores stock metadata with an ObjectId reference linking each stock to its sector document.
stock_prices	_id, stock_id, trade_date, open/high/low/close_price, volume	stock_id → stocks._id	The main time-series collection. Each document represents one trading day for one stock, with an ObjectId reference to the stocks collection.

Table 7: MongoDB collection descriptions and references

C. Database Queries (PostgreSQL + MongoDB)

The table below lists the five equivalent queries we implemented for both databases. They cover the required latest-record and date-range patterns, plus three analytical queries that show what each database can do with time-series data.

#	Query Description	PostgreSQL Feature	MongoDB Feature
1	Latest closing price per stock: retrieves the most recent trading record for each of the five tickers.	Correlated subquery using MAX(trade_date) per stock_id	$sort → $group → $lookup aggregation pipeline
2	Date-range query: all AAPL prices within the year 2017 (first 5 rows).	WHERE trade_date BETWEEN '2017-01-01' AND '2017-12-31'	$match with $gte / $lte operators on trade_date

3	7-day moving average of the closing price for JPM: demonstrates rolling window analytics.	AVG() OVER (ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)	$setWindowFields with documents: [-6, 0]
4	Annual average, maximum, and minimum closing price per stock: summarises yearly price behaviour.	GROUP BY ticker + EXTRACT(YEAR FROM trade_date)	$group stage with $year date operator
5	Most volatile stock by standard deviation of daily closing price: identifies AAPL as most volatile.	STDDEV_SAMP() aggregate, ORDER BY std_dev DESC	$stdDevSamp accumulator + $lookup for company name

Table 8: Query descriptions and implementation approaches for both databases

Task 3: CRUD Endpoints and Time-Series Queries

A. API Architecture

The API is built with FastAPI and organised into four layers within the src/ directory. src/main.py initialises the FastAPI application and manages the Prisma database connection lifecycle using an async lifespan context manager. Route definitions are in src/api/router.py, business logic is in src/services/timeseries_service.py, and Pydantic request and response validation models are in src/schemas/stock_prices.py.

File	Responsibility
src/main.py	FastAPI app entry point; connects and disconnects the Prisma client on startup and shutdown.
src/api/router.py	Defines all five CRUD route handlers and maps them to HTTP methods and URL paths.
src/services/timeseries_service.py	Contains the TimeseriesService class with business logic for query construction and ORM calls.
src/schemas/stock_prices.py	Pydantic models: StockPriceCreate, StockPriceUpdate, StockPriceResponse, StockPriceFilter.
src/database/clients.py	Instantiates and exports the Prisma PostgreSQL client used across the application.

Table 9: API source files and their responsibilities

B. Endpoints

The API exposes five endpoints that together provide full CRUD capability for the stock_prices time-series records stored in PostgreSQL.

Method	Endpoint	Description	HTTP Status
POST	/api/stock-prices/	Creates a new daily price record. Validates the payload against StockPriceCreate.	201 Created
GET	/api/stock-prices/	Returns a filtered, paginated list of price records. Supports 8 query filter parameters.	200 OK
GET	/api/stock-prices/{price_id}	Fetches a single price record by its primary key (price_id).	200 OK
PUT	/api/stock-prices/{price_id}	Updates one or more fields of an existing record. Only provided fields are changed.	200 OK
DELETE	/api/stock-prices/{price_id}	Permanently removes a record from the database. Returns no body on success.	204 No Content

Table 10: CRUD endpoint definitions

C. Query Filters and Time-Series Endpoints

The GET list endpoint filters by stock_id, ticker (joined to the stocks table), start_date, end_date, min_price, max_price, min_volume, and max_volume, with skip/limit pagination. Results come back ordered by trade_date descending, so limit=1 always returns the most recent record for a stock. That covers the required 'latest record' endpoint, and combining start_date and end_date covers the 'records by date range' requirement.

D. Service Layer and Error Handling

TimeseriesService.get_stock_prices() builds a dynamic Prisma where clause from the StockPriceFilter object. Ticker filters are translated into a relational join on the stocks table. Date, price, and volume filters use Prisma's gte and lte conditions. Standard HTTP error codes are returned: 404 Not Found for missing records, 400 Bad Request for invalid payloads, and 500 Internal Server Error for unexpected failures.

E. API Request Flow

The table below walks through the lifecycle of a typical create request and a filtered GET request across the API layers.

Step	Actor	Action
1	Client	POST /api/stock-prices/ with JSON payload
2	FastAPI	Validates payload against StockPriceCreate Pydantic schema
3	Service	create_stock_price(data) is called on TimeseriesService
4	Prisma/DB	INSERT INTO stock_prices ... executed against PostgreSQL
5	API→Client	201 Created response with StockPriceResponse body

6	Client	GET /api/stock-prices/?ticker=AAPL&start_date=2017-01-01
7	Service	Builds dynamic where clause: ticker join + date gte/lte
8	Prisma/DB	SELECT with stocks JOIN, date filter, ORDER BY trade_date DESC
9	API→Client	200 OK with list of StockPriceResponse objects

Table 11: API request flow for POST and GET operations

F. Example API Calls

Fetch latest record

```
GET /api/stock-prices/?ticker=AAPL&limit=1
```

Fetch records by date range

```
GET /api/stock-prices/?ticker=AAPL&start_date=2017-01-
01&end_date=2017-12-31
```

Create a record

```
POST /api/stock-prices/
{ "stock_id": 1, "trade_date": "2017-06-15", "open_price":
145.77,
  "high_price": 148.10, "low_price": 145.12, "close_price":
147.82, "volume": 21300000 }
```

Task 4: Prediction / Forecast Script

Task 4 ties the three earlier tasks together into one prediction pipeline. Instead of a standalone script, the forecast runs as its own API endpoint (POST /api/predict/) inside the FastAPI application. That way the prediction service can be called alongside the CRUD endpoints, and the whole pipeline is reachable over HTTP.

A. Overview and Architecture

The prediction pipeline is implemented across three new files added to the src/ directory, alongside a saved model artefact in the model/ folder. The updated src/main.py registers the prediction router alongside the existing stock prices router, so both CRUD and prediction endpoints are served by the same FastAPI application.

File	Role
src/api/prediction.py	Defines the POST /api/predict/ endpoint. Accepts a PredictionRequest payload, delegates to PredictionService, and returns a PredictionResponse.

src/schemas/prediction.py	Pydantic models for request validation (PredictionRequest) and response serialisation (PredictionResponse). Enforces all 15 required input features.
src/services/prediction_service.py	Loads the saved Linear Regression model and feature order from disk on first call (singleton pattern). Constructs a DataFrame from the input, aligns feature columns to the training order, and returns the predicted closing price.
model/linear_regression_model.pkl	The trained Linear Regression model serialised with joblib, saved from the Task 1 notebook.
model/feature_order.pkl	A pickled list that records the exact column order used during training, so features always reach the model in the correct sequence.

Table 13: Task 4 source files and their roles

B. Prediction Endpoint

The prediction endpoint is registered at POST /api/predict/ and accepts a JSON payload matching the PredictionRequest schema. It returns the predicted next-day closing price as a single float value wrapped in a PredictionResponse object.

```
POST /api/predict/
{
  "Open":      145.77,   // Opening price for the trading day
  "High":      148.10,   // Highest price during the trading
day
  "Low":       145.12,   // Lowest price during the trading
day
  "Volume":    21300000, // Trading volume
  "Lag1":      144.50,   // Previous day closing price
  "MA7":       143.80,   // 7-day moving average of closing
price
  "Return":    0.0088,   // Daily percentage return
  "Year":      2017,     // Calendar year
  "Month":     6,        // Calendar month
  "Day":       15,       // Calendar day
  "DayOfWeek": 3,        // Day of week (0=Monday, 4=Friday)
  "Name_CAT":  0,        // 1 if Caterpillar, else 0
  "Name_JPM":  0,        // 1 if JPMorgan Chase, else 0
  "Name_MCD":  0,        // 1 if McDonalds, else 0
  "Name_WMT":  0         // 1 if Walmart, else 0 (AAPL = all
zeros)
}
```

The response returns the predicted closing price:

```
{ "prediction": 147.23 }
```

C. Prediction Service: Step-by-Step Pipeline

The PredictionService class implements the four steps required by Task 4:

Step	Action	Implementation Detail
A: Fetch	Retrieve stock price data from the API	The client calls GET /api/stock-prices/?ticker=AAPL&limit=1 to retrieve the latest record, then extracts the required fields to construct the PredictionRequest payload.
B: Preprocess	Apply the same feature engineering as Task 1	The PredictionRequest schema enforces all 15 features used during training: Open, High, Low, Volume, Lag1, MA7, Return, Year, Month, Day, DayOfWeek, and the four one-hot ticker dummies. The service builds a single-row DataFrame and reorders columns using the saved feature_order.pkl to match the training layout exactly.
C: Load model	Load the trained Linear Regression model	PredictionService uses joblib.load() to deserialise model/linear_regression_model.pkl on the first request. A singleton pattern ensures the model is loaded only once and reused across all subsequent requests.
D: Predict	Generate the next-day closing price forecast	model.predict(X)[0] is called on the aligned feature DataFrame. The result is cast to float and returned as the prediction field of the PredictionResponse.

Table 14: Task 4 pipeline steps and implementation details

D. Integration with the API

The prediction router is registered in src/main.py next to the stock prices router, so the full pipeline runs inside the same FastAPI application:

```
app.include_router(stock_prices_router)  # CRUD endpoints
app.include_router(prediction_router)   # Prediction endpoint
```

So a client can pull the latest record from GET /api/stock-prices/, take the fields it needs, and post them straight to POST /api/predict/. That completes the full fetch, preprocess, predict cycle in two HTTP calls, with no offline scripting.

E. End-to-End Flow

Step	Actor	Action
1	Client	GET /api/stock-prices/?ticker=AAPL&limit=1
2	API	Returns the most recent AAPL price record (trade_date, open, high, low, close, volume)

3	Client	Computes Lag1, MA7, Return, calendar features from the fetched record
4	Client	POST /api/predict/ with the 15-feature JSON payload
5	FastAPI	Validates payload against PredictionRequest Pydantic schema
6	PredictionService	Builds DataFrame, aligns feature order from feature_order.pkl
7	LR Model	model.predict(X) returns the next-day predicted closing price
8	API→Client	200 OK: { "prediction": 147.23 }

Table 15: End-to-end prediction

Implementation Notes

A. Technology Stack

Layer	Technology	Notes
Language	Python 3.11+	
API Framework	FastAPI	Async, type-safe REST API with automatic OpenAPI docs
ORM	Prisma Python v0.15.0	Schema: config/database/postgres/prisma/schema.prisma
SQL Database	PostgreSQL	Primary runtime database for all CRUD operations
NoSQL Database	MongoDB	Design + seed scripts present; API currently wired to PostgreSQL
ML Libraries	scikit-learn	RandomForestRegressor, LinearRegression, GridSearchCV
Data Processing	pandas / numpy	Feature engineering, preprocessing, train/test split
Env Management	pipenv + .env	POSTGRES_DATABASE_URL, MONGODB_DATABASE_URL

Table 12: Technology stack

B. Repository Structure

```
ml-pipeline-formative-1/
├── config/database/postgres/   # Prisma schema + seed.sql + ERD diagram
```

```
├── config/database/mongo/      # MongoDB schema + seed.js
├── dataset/                    # Raw CSV files (5 tickers +
combined dataset)
├── model/                      # Saved model artefacts
├── notebook/ML_Pipeline_Formative1.ipynb  # Task 1 EDA +
modelling
├── scripts/                    # DB loaders, query scripts,
forecast_stock.py
├── src/                        # FastAPI app (main, router,
service, schemas)
├── .env.example  |  Pipfile  |  README.md
```

References

Atsalakis, G. S. and Valavanis, K. P. (2009). Surveying stock market forecasting techniques - Part II: Soft computing methods. Expert Systems with Applications, 36(3), 5932-5941.

Box, G. E. P. and Jenkins, G. M. (1976). Time Series Analysis: Forecasting and Control. San Francisco: Holden-Day.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. The Journal of Finance, 25(2), 383-417.

Fowler, M. and Sadalage, P. J. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley.

Khaidem, L., Saha, S. and Dey, S. R. (2016). Predicting the direction of stock market prices using random forest. arXiv preprint arXiv:1605.00003.

Murphy, J. J. (1999). Technical Analysis of the Financial Markets: A Comprehensive Guide to Trading Methods and Applications. New York Institute of Finance.

Prado, M. L. de (2018). Advances in Financial Machine Learning. Wiley.

Ramalho, L. (2021). Fluent Python: Clear, Concise, and Effective Programming (2nd ed.). O'Reilly Media.

Kaggle (2024). Stock Market Dataset. Available at: https://www.kaggle.com/datasets/szrlee/stock-time-series-20050101-to-20171231?select=AAPL_2006-01-01_to_2018-01-01.csv